COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 - Data Structures and Algorithms	**SECTION:**	CPE21S2
DATE PERFORMED:	10/2/26	**DATE SUBMITTED:**	10/2/26
NAME:	Macasaet John David C.	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. _10__ Graphs

1. PROCEDURE OUTPUT

Code

```cpp
#include <iostream>
// stores adjacency list items
struct adjNode {
    int val, cost;
    adjNode* next;
};
// structure to store edges
struct graphEdge {
    int start_ver, end_ver, weight;
};

class DiaGraph{
// insert new nodes into adjacency list from given graph
adjNode* getAdjListNode(int value, int weight, adjNode* head){
    adjNode* newNode = new adjNode;
    newNode->val = value;
    newNode->cost = weight;
    newNode->next = head; // point new node to current head
    return newNode;
  }


int N; //number of node in the graph

public:
adjNode **head; //adjacency list as array of pointers
// Constructor
DiaGraph(graphEdge edges[], int n, int N) {
    // allocate new node
    head = new adjNode*[N]();
    this->N = N;
    // initialize head pointer for all vertices
    for (int i = 0; i < N; ++i){
        head[i] = nullptr;
    // construct directed graph by adding edges to it
    }
    for (unsigned i = 0; i < n; i++) {
        int start_ver = edges[i].start_ver;
        int end_ver = edges[i].end_ver;
        int weight = edges[i].weight;
        // insert in the beginning
        adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);
        // point head pointer to new node
        head[start_ver] = newNode;
```

```
    }
}
// Destructor
~DiaGraph() {
    for (int i = 0; i < N; i++)
        delete[] head[i];
        delete[] head;
    }
};

// print all adjacent vertices of given vertex
void display_AdjList(adjNode* ptr, int i){
    while (ptr != nullptr) {
       std::cout << "(" << i << ", " << ptr->val
          << ", " << ptr->cost << ") ";
       ptr = ptr->next;
    }
    std::cout << std::endl;
}
// graph implementation
int main(){
    // graph edges array.
    graphEdge edges[] = {
    // (x, y, w) -> edge from x to y with weight w
    {0,1,2},{0,2,4},{1,4,3},{2,3,2},{3,1,4},{4,3,3}
    };
    int N = 6; // Number of vertices in the graph
    // calculate number of edges
    int n = sizeof(edges)/sizeof(edges[0]);
    // construct graph
    DiaGraph diagraph(edges, n, N);
    // print adjacency list representation of graph
    std::cout<<"Graph adjacency list "<<std::endl<<"(start_vertex, end_vertex, weight):"<<std:
    for (int i = 0; i < N; i++){
        // display adjacent vertices of vertex i
        display_AdjList(diagraph.head[i], i);
    }
    return 0;
}
```

Output

```
Graph adjacency list
(start_vertex, end_vertex, weight):
(0, 2, 4) (0, 1, 2)
(1, 4, 3)
(2, 3, 2)
(3, 1, 4)
(4, 3, 3)

PS C:\Users\TIPQC\Downloads\HOA 10>
```

Table 1: ILO A

Code

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>


template <typename T>
class Graph;

template <typename T>
struct Edge{
    size_t src;
    size_t dest;
    T weight;
    // To compare edges, only compare their weights,
    // and not the source/destination vertices
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++){
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
            os << std::endl;
    }
    return os;
}

template <typename T>
class Graph{
public:
```

```
    // Initialize the graph with N vertices
    Graph(size_t N) : V(N)
    {
    }
    // Return number of vertices in the graph
    auto vertices() const
    {
    return V;
    }
    // Return all edges in the graph
    auto &edges() const
    {
    return edge_list;
    }
    void add_edge(Edge<T> &&e)
    {
        // Check if the source and destination vertices are within range
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
    // Returns all outgoing edges from vertex v
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
               edges_from_v.emplace_back(e);
        }
        return edges_from_v;
    }
    // Overloads the << operator so a graph be written directly to a stream
    // Can be used as std::cout << obj << std::endl;
    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V; // Stores number of vertices in graph
    std::vector<Edge<T>> edge_list;
```

```
};

template <typename T>
auto depth_first_search(const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1); // Assume that DFS always starts from vertex ID 1
    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                // If the vertex hasn't been visited, insert it in the stack.
                if(visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}

template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
```

```
129     edges[7] = {{3, 0}, {6, 0}};
130     edges[8] = {{4, 0}, {5, 0}, {6, 0}};
131     for (auto &i : edges)
132         for (auto &j : i.second)
133             G.add_edge(Edge<T>{i.first, j.first, j.second});
134     return G;
135 }
136
137 template <typename T>
138 void test_DFS()
139 {
140     // Create an instance of and print the graph
141     auto G = create_reference_graph<unsigned>();
142     std::cout << G << std::endl;
143     // Run DFS starting from vertex ID 1 and print the order
144     // in which vertices are visited.
145     std::cout << "DFS Order of vertices: " << std::endl;
146     auto dfs_visit_order = depth_first_search(G, 1);
147     for (auto v : dfs_visit_order)
148         std::cout << v << std::endl;
149     }
150 int main()
151 {
152     using T = unsigned;
153     test_DFS<T>();
154     return 0;
155 }
```

Output

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
PS C:\Users\TIPQC\Downloads\HOA 10>
```

Table 2: ILOB1: DFS

Code

```cpp
#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;
    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator>(const Edge<T> &e) const
    {
        return this->weight > e.weight;
    }
};

template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
```

```
    for (auto i = 1; i < G.vertices(); i++)
    {
        os << i << ":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
            os << "{" << e.dest << ": " << e.weight << "}, ";
        os << std::endl;
    }
    return os;
};

template <typename T>
class Graph
{
public:
    Graph(size_t N) : V(N) {}
    auto vertices() const
    {
        return V;
    }
    auto &edges() const
    {
        return edge_list;
    }
    void add_edge(Edge<T> &&e)
    {
        if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            edge_list.emplace_back(e);
        else
            std::cerr << "Vertex out of bounds" << std::endl;
    }
    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
```

```
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};
    for (auto &i : edges)
        for (auto &j : i.second)
            G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}

template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1); // Assume that BFS always starts from vertex ID 1
    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();
        // If the current vertex hasn't been visited in the past
        if (visited.find(current_vertex) == visited.end())
```

```
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
                queue.push(e.dest);
        }
    }
    return visit_order;
}
template <typename T>
void test_BFS(){
    // Create an instance of and print the graph
    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;
    // Run BFS starting from vertex ID 1 and print the order
    // in which vertices are visited.
    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);
    for (auto v : bfs_visit_order)
        std::cout << v << std::endl;
    }
int main()
{
    using T = unsigned;
    test_BFS<T>();
    return 0;
```

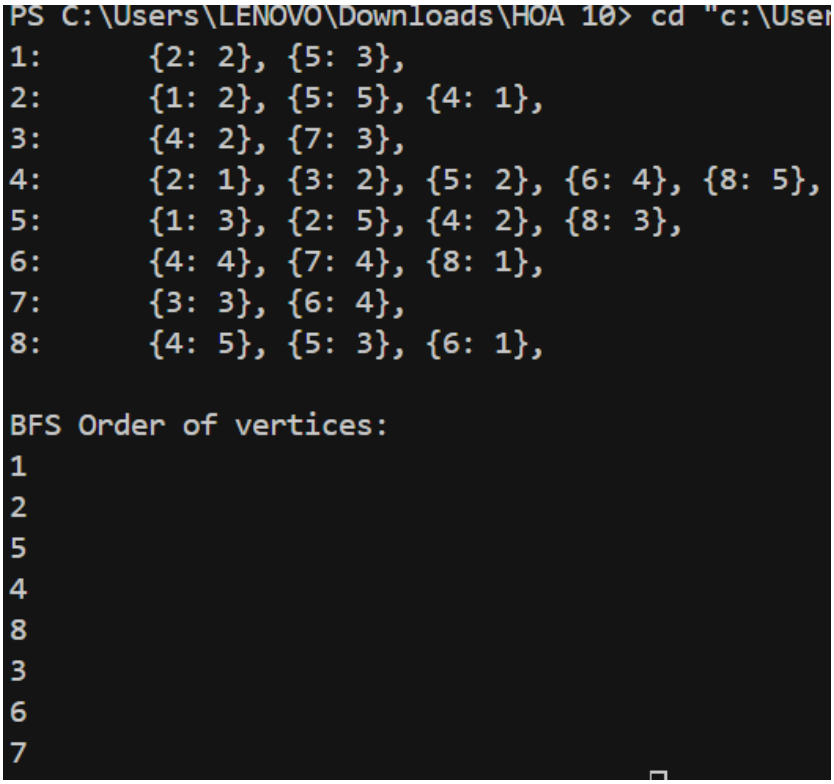
Output	

Table 3: ILOB2: BFS

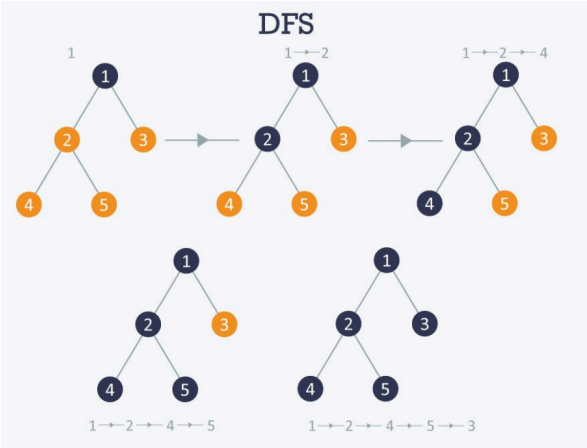
Observation: BFS and DFS use different data structures to explore nodes. BFS uses a queue (FIFO) to visit nodes layer by layer without backtracking. DFS uses a stack (LIFO) where new nodes are added to the top. By using pop(), DFS visits the newest node at the top to go deeper into a path. Once it hits a dead end, continuing to pop() takes you back to older unvisited paths, which is how it backtracks.

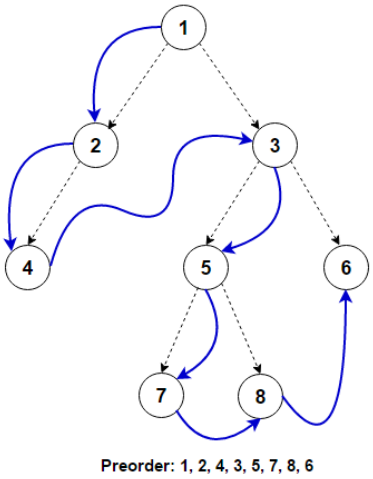
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from the same vertex. Discuss which algorithm would be most helpful to accomplish this task.
_Depth-first Search algorithm is the most useful for this problem since it allows for backtracking and when you start from one vertex it will traverse through its adjacent vertex and the adjacent vertex of that vertex and so on until it reaches the bottom or the end. And then it backtracks so that it will visit the other vertices not yet visited. This algorithm mirrors the problem shown.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.
____DFS is the most equivalent to Pre-Order traversal since looking at the graph provided below for DFS it shows a Root-Left-Right order wherein it starts at one node and goes to the left until it reaches the bottom left and then goes to the right. Both allow for backtracking, in Pre-order if the left side reaches nullptr then it move to the top

right and in DFS, it uses pop() to backtrack.





Preorder: 1, 2, 4, 3, 5, 7, 8, 6

3. In the performed code, what data structure is used to implement the Breadth First Search?
___In BFS, a queue is used for this algorithm. It uses this data structure because it does follow a forward path of traversing layer by layer instead of needing to backtrack like in DFS. Queue is perfect for this since it follows a FIFO rule in which the ones in front will be visited first followed by the rest until all has been visited. This is in line with the layer by layer visiting of BFS where you start from the first layer and traverse to the next layers.

4. How many times can a node be visited in the BFS?
__In BFS, nodes can only be visited once since it does not require backtracking and you only move from one vertex to another layer by layer until all have been visited. BFS visits vertices in a way that if the node is considered visited then there is no way to go back again.

CONCLUSION: A graph is a data structure made of vertices connected by edges used to represent networks like maps or relationships. Graph traversal relies heavily on the underlying data structure used. Breadth-First Search uses a queue to traverse nodes layer-by-layer, making it ideal for finding the shortest path without backtracking. In contrast, Depth-First Search uses a stack or recursion to explore as deep as possible along each branch before using pop to backtrack and explore remaining paths.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGpt**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use:

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.